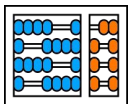




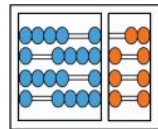
Prática com os Kits

Apresentação dos Kits e configuração do Linux

Mateus Antonio da Silva - Instituto de Pesquisas Eldorado



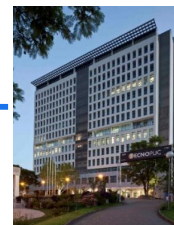
Instituto de Pesquisas Eldorado



Manaus



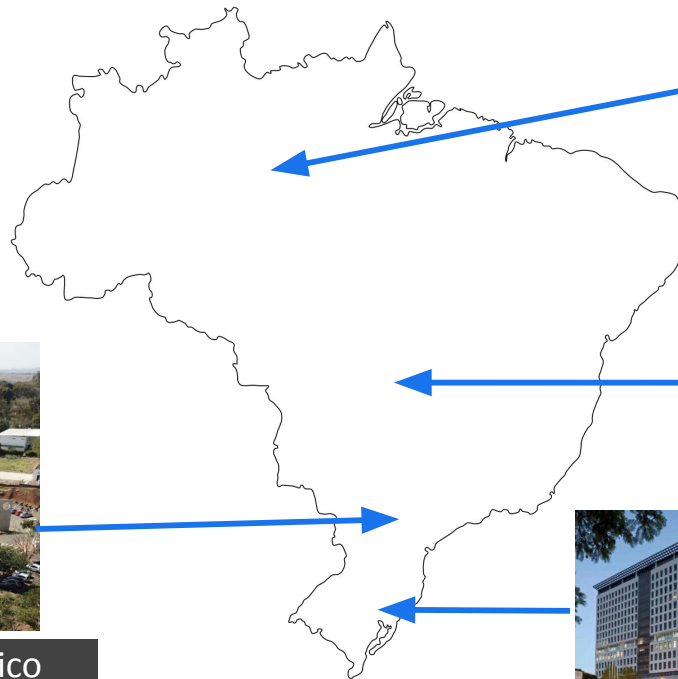
Brasília



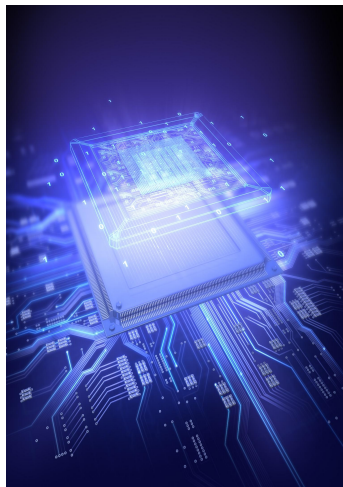
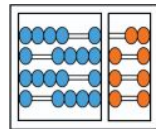
Porto Alegre



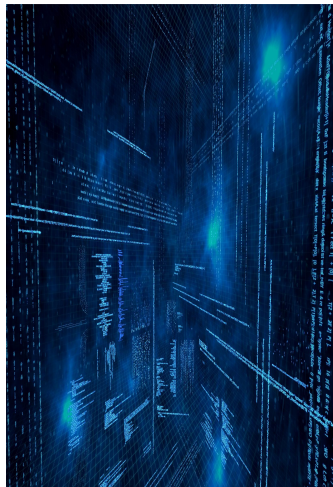
Campinas – Parque Tecnológico
UNICAMP



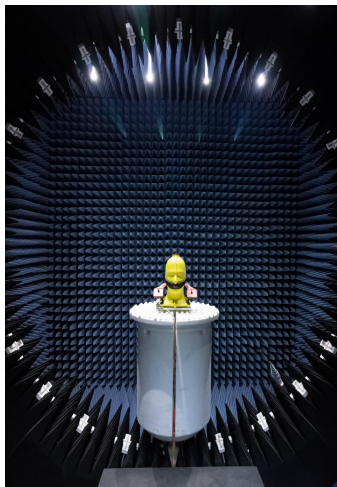
Instituto de Pesquisas Eldorado - Áreas de Atuação



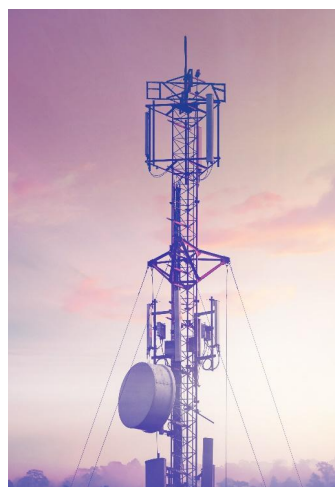
Soluções em Hardware e Software



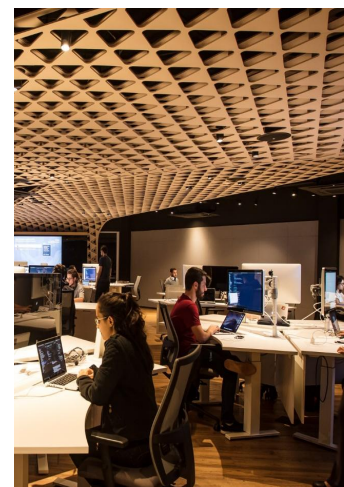
Consultoria em Inovação



Testes de Laboratório

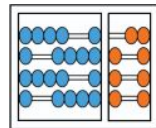


Certificação de Produtos Telecom



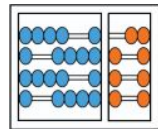
Capacitação e Treinamento

Participação do Eldorado

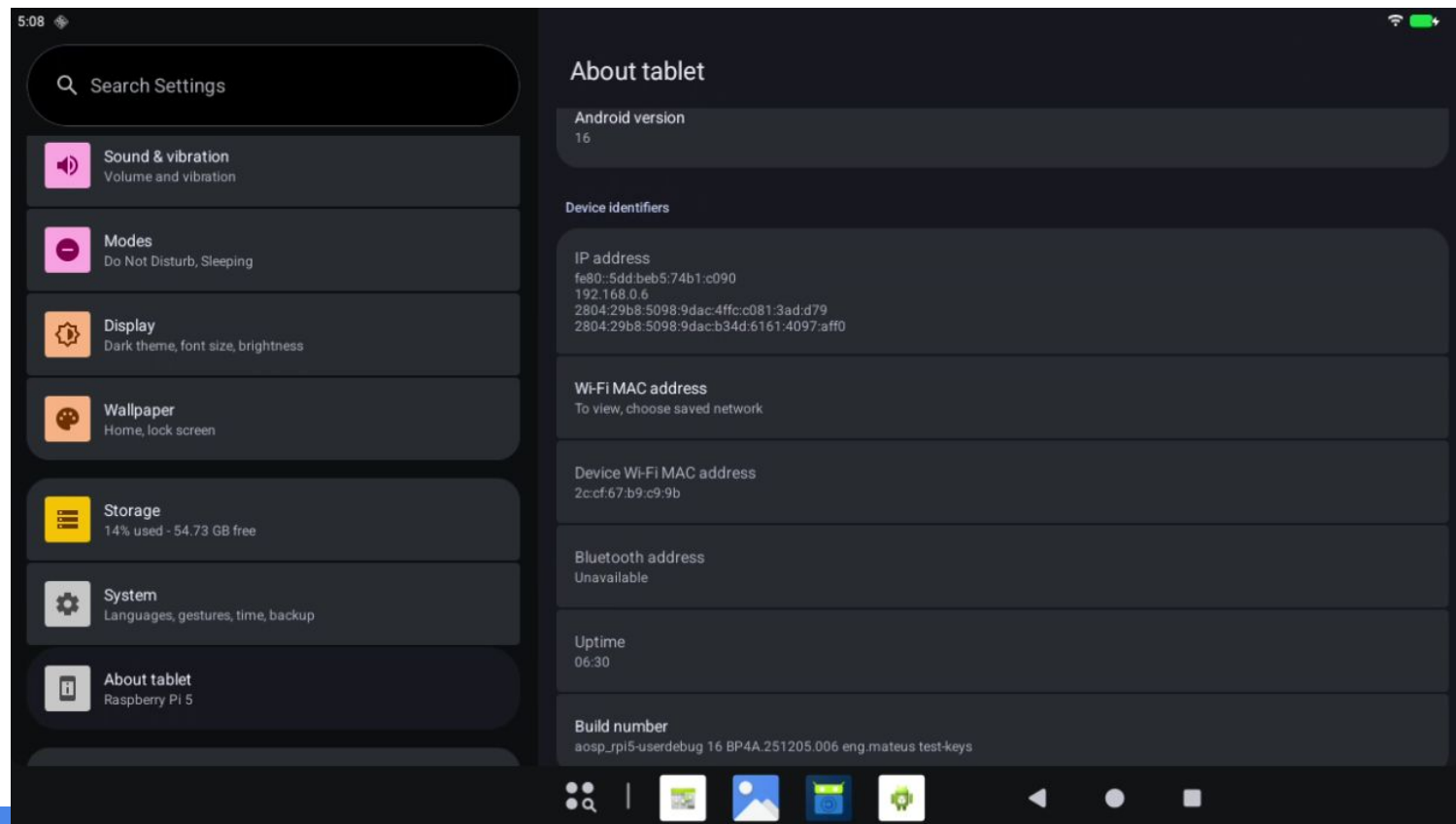
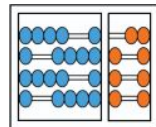


- [Coordenadora] Raquel Pignatelli da Silva
 - Contato: raquel.silva@eldorado.org.br
- [Mentor Técnico] Mateus Antonio da Silva
 - Engenheiro da Computação - UFPB
 - Mestrando no IC - UNICAMP
 - Analista de Software - Instituto de Pesquisas Eldorado
 - [2021-2025] Projeto Motorola
 - [2025-Atual] Projetos Educacionais
 - Interesses: Impressão 3D, Drones, Sistemas Embarcados, IoT, etc...
 - Contato: mateusa@eldorado.org.br

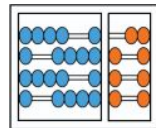
Sobre o Kit



Sobre o Kit - SO Instalado



Sobre o Kit - Lista de Componentes



Placa Principal	Raspberry Pi 5 Modelo B - Rev 1.1 (8GB)
Módulo de Sensores	Sense HAT - Rev 2
Tela sensível a toque (7 polegadas)	Raspberry Pi Touch Display 2
2x Cartão de Memória	Cartão de Memória MicroSD 64GB Classe A2
Fonte de Alimentação	USB-C Oficial 5V/5A
Caixa de proteção	Case para Raspberry Pi 5 (Vermelho/Branco)

2x Espaçador	Espaçador M2.5 de Latão
Barramento de pinos	Barramento e extensor de Fios Extended GPIO Female Header - 2x20 Pin (13.5mm/9.80mm)
Parafusos	Parafusos M2.5
2x Arruela	Arruela para espaçador
Base do kit	Suporte para montagem do Kit completo; Fixação na Tela Touch
Adaptador MicroSD para USB	

Protocolo Inicial do RPi 5 Kit & IoT:

Diretrizes Essenciais para a Longevidade e Desempenho Máximo

Garanta a estabilidade e o potencial da sua estação de desenvolvimento embarcado.



O Conceito (Teoria Relâmpago)

Este kit não é apenas um computador de placa única; é uma pilha de hardware integrada (RPi 5, Display Touch 2 e Sense HAT) exigindo cuidado coordenado.

A arquitetura de alto desempenho do Raspberry Pi 5 demanda gerenciamento térmico ativo e energia robusta via USB-PD para operar sem throttling.

O manuseio inadequado pode comprometer a integridade dos sinais no barramento GPIO conectando o Sense HAT ou deformar a estrutura de suporte.



Bancada de Trabalho (Requisitos)

-  **RPi 5 Kit Montado** (Unidade central RPi 5, acoplada ao Display Touch 2, com Sense HAT no topo).
-  **Base de suporte impressa em 3D** (instalada).
-  **Fonte de Alimentação Oficial Raspberry Pi** (27W USB-C PD).
-  **Cartão SD** (com imagem RPi OS ou AOSP gravada).
-  **Superfície de trabalho limpa, plana e livre de estática (ESD).**



1



Inspeção da Base 3D e Ambiente

A base é de termoplástico (PLA/PETG).
Cuidado Crítico: Nunca exponha o kit à luz solar direta ou dentro de veículos quentes; o calor deformará irreversivelmente a base, desalinhando o display. Mantenha sempre em ambiente ventilado.

2



Manuseio da Pilha de PCB

Ao mover o kit, segure sempre pelas bordas da base 3D ou pelas laterais do display. Evite tocar diretamente nos componentes expostos do Sense HAT ou na placa RPi 5 para prevenir danos por descarga eletrostática (ESD) aos sensores sensíveis.

3



Gerenciamento Térmico (O Fator Pi 5)

O RPi 5 gera significativamente mais calor que seus antecessores. O fluxo de ar é vital durante compilações Android pesadas.



Passo a Passo (Protocolo de Execução)



O RPi 5 gera significativamente mais calor que seus antecessores.
O fluxo de ar é vital durante compilações Android pesadas.

4



Procedimento de Ligação

- Insira o cartão SD no slot inferior.
- Conecte o adaptador de energia à porta USB-C. O kit iniciará automaticamente.

Nota: Verifique se o LED de status (verde) começa a piscar, indicando leitura do cartão SD.

5



Procedimento de Desligamento Seguro

- Nunca puxe o cabo de energia enquanto o sistema estiver em execução. Isso corrompe o sistema de arquivos (ext4/f2fs).



No RPi OS:

Menu Iniciar -> Desligar.



No Android/AOSP:

Use o menu de energia na interface ou o comando via ADB shell: **reboot -p**.

Aguarde o LED verde parar de piscar completamente e o LED vermelho (energia) ficar estático antes de remover a fonte de alimentação.

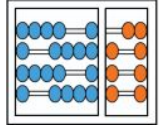


Atenção

A Ditadura da Amperagem: O Raspberry Pi 5 negocia energia ativamente. Se você usar um carregador de celular padrão, o sistema detectará a falta de capacidade de corrente (menos de 5A a 5V), exibirá um 'Aviso de Subtensão' e limitará severamente o desempenho da CPU e a corrente da porta USB. Para projetos de IoT usando o Sense HAT e o Display simultaneamente, usar a fonte de alimentação oficial de 27W não é opcional; é obrigatório para a estabilidade dos sensores.

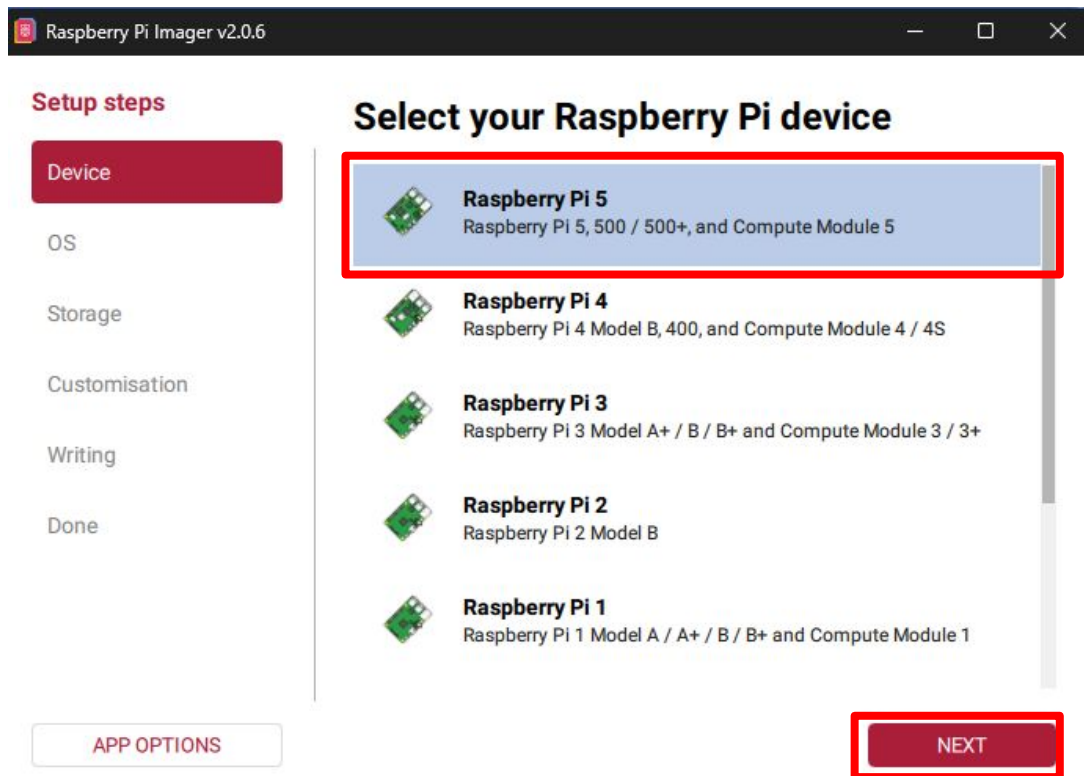
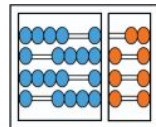
Prática - Configurando o RPI OS

Download do Raspberry Pi Imager

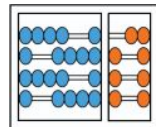


- Fontes:
 - <https://www.raspberrypi.com/software/>
 - <https://github.com/raspberrypi/rpi-imager/releases>

Passo 1: Escolher o dispositivo



Passo 2: Selecionar o OS



Raspberry Pi Imager v2.0.6

Setup steps

Device

OS

Storage

Customisation

Writing

Done

Choose operating system

Select an operating system to install on your Raspberry Pi

**Raspberry Pi OS (64-bit)**

A port of Debian Trixie with the Raspberry Pi Desktop (Recommended)
Online - 1.2 GB download
Released: 2025-12-04

**Raspberry Pi OS (32-bit)**

A port of Debian Trixie with the Raspberry Pi Desktop
Online - 1.2 GB download
Released: 2025-12-04

**Raspberry Pi OS (Legacy, 32-bit)**

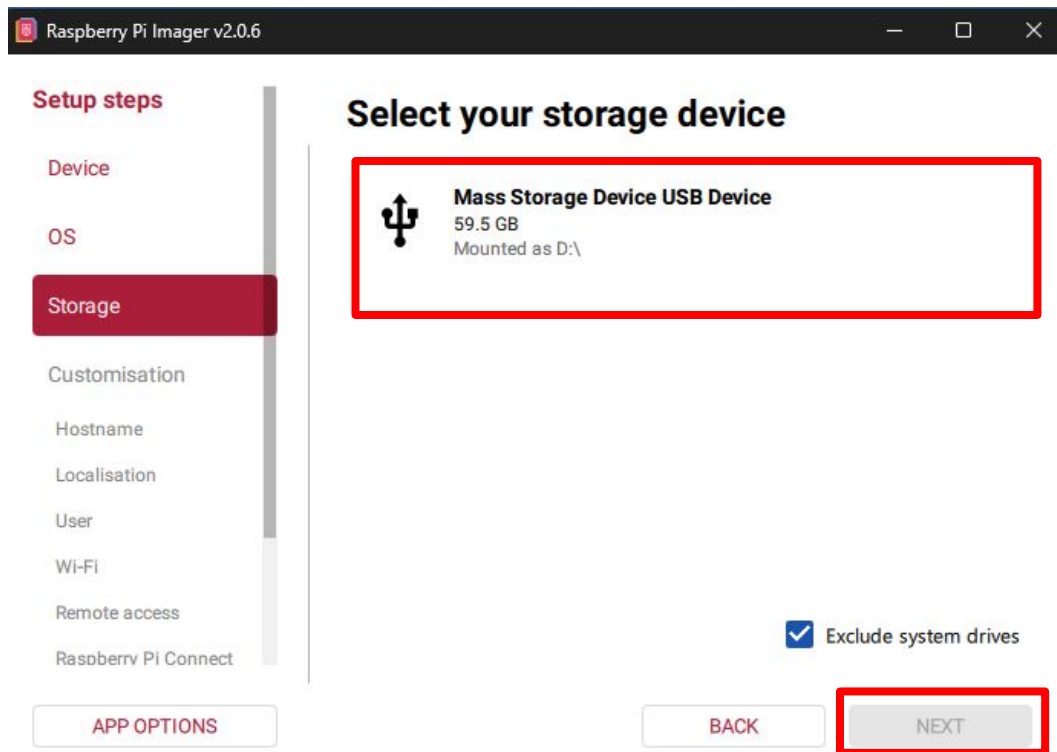
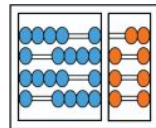
A port of Debian Bookworm with security updates and desktop environment
Online - 1.1 GB download
Released: 2025-11-24

APP OPTIONS

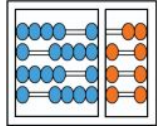
BACK

NEXT

Passo 3: Selecionar o cartão de memória



Passo 4: Inserir hostname



Raspberry Pi Imager v2.0.6

Device

OS

Storage

Customisation

Hostname

Localisation

User

Wi-Fi

Remote access

Raspberry Pi Connect

Writing

Customisation: Choose hostname

Enter your hostname

pi

A hostname is a unique name that identifies your Raspberry Pi on the network. It should contain only letters, numbers, and hyphens.

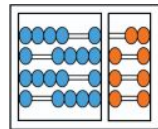
APP OPTIONS

SKIP CUSTOMISATION

BACK

NEXT

Passo 5: Configurar localização



Raspberry Pi Imager v2.0.6

OS

Storage

Customisation

Hostname

Localisation

User

Wi-Fi

Remote access

Raspberry Pi Connect

Writing

Customisation: Localisation

Select your location for suggested time zone and keyboard layout

Capital city: Brasília (Brazil)

Time zone: America/Sao_Paulo

Keyboard layout: br

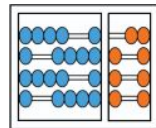
APP OPTIONS

SKIP CUSTOMISATION

BACK

NEXT

Passo 6: Configurar usuário



Raspberry Pi Imager v2.0.6

OS

Storage

Customisation

Hostname

Localisation

User

Wi-Fi

Remote access

Raspberry Pi Connect

Writing

Done

Customisation: Choose username

Create a user account for your Raspberry Pi

Username:

Password:

Confirm password:

The username must be lowercase and contain only letters, numbers, underscores, and hyphens.

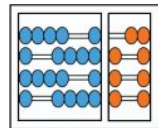
APP OPTIONS

SKIP CUSTOMISATION

BACK

NEXT

Passo 7: Configurar Wi-Fi



Raspberry Pi Imager v2.0.6

Storage

Customisation

Hostname

Localisation

User

Wi-Fi

Remote access

Raspberry Pi Connect

Writing

Done

Customisation: Choose Wi-Fi

SECURE NETWORK OPEN NETWORK

SSID: Network name
network_name

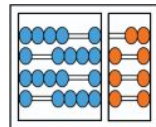
Password: Network password
.....

Confirm password: Re-enter password
.....

☐ Hidden SSID

APP OPTIONS SKIP CUSTOMISATION BACK **NEXT**

Passo 8: Configurar SSH



Raspberry Pi Imager v2.0.6

Storage

Customisation

Hostname

Localisation

User

Wi-Fi

Remote access

Raspberry Pi Connect

Writing

Done

Customisation: SSH authentication

Configure SSH access

Enable SSH ☒

[Learn about SSH](#)

Authentication mechanism:

☒ Use password authentication

☐ Use public key authentication

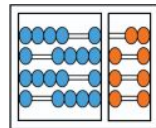
APP OPTIONS

SKIP CUSTOMISATION

BACK

NEXT

Passo 9: Configurar RPI Connect



Raspberry Pi Imager v2.0.6

Storage

Customisation

Hostname

Localisation

User

Wi-Fi

Remote access

Raspberry Pi Connect

Writing

Done

Customisation: Raspberry Pi Connect

Sign in to receive a token and enable Raspberry Pi Connect

Enable Raspberry Pi Connect
What is Raspberry Pi Connect?

☐

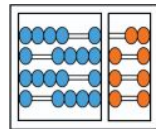
APP OPTIONS

SKIP CUSTOMISATION

BACK

NEXT

Passo 10: Revisar configurações



Device

OS

Storage

Customisation

Hostname

Localisation

User

Wi-Fi

Remote access

Raspberry Pi Connect

Writing

Write image

Review your choices and write the image to the storage device

Summary

Device: **Raspberry Pi 5**
Operating system: **Raspberry Pi OS (64-bit)**
Storage: **Mass Storage Device USB Device**

Customisations to apply:

- Hostname configured
- Localisation configured
- User account configured
- Wi-Fi configured
- SSH enabled

APP OPTIONS

BACK

WRITE



Storage

Customisation

Hostname

Localisation

User

Wi-Fi

Remote access

Raspberry Pi Connect

Writing

Done

Write complete!

Your choices:

Device: **Raspberry Pi 5**
Operating system: **Raspberry Pi OS (64-bit)**
Storage: **Mass Storage Device USB Device**

Customisations applied:

- ✓ Hostname configured
- ✓ Localisation configured
- ✓ User account configured
- ✓ Wi-Fi configured
- ✓ SSH enabled

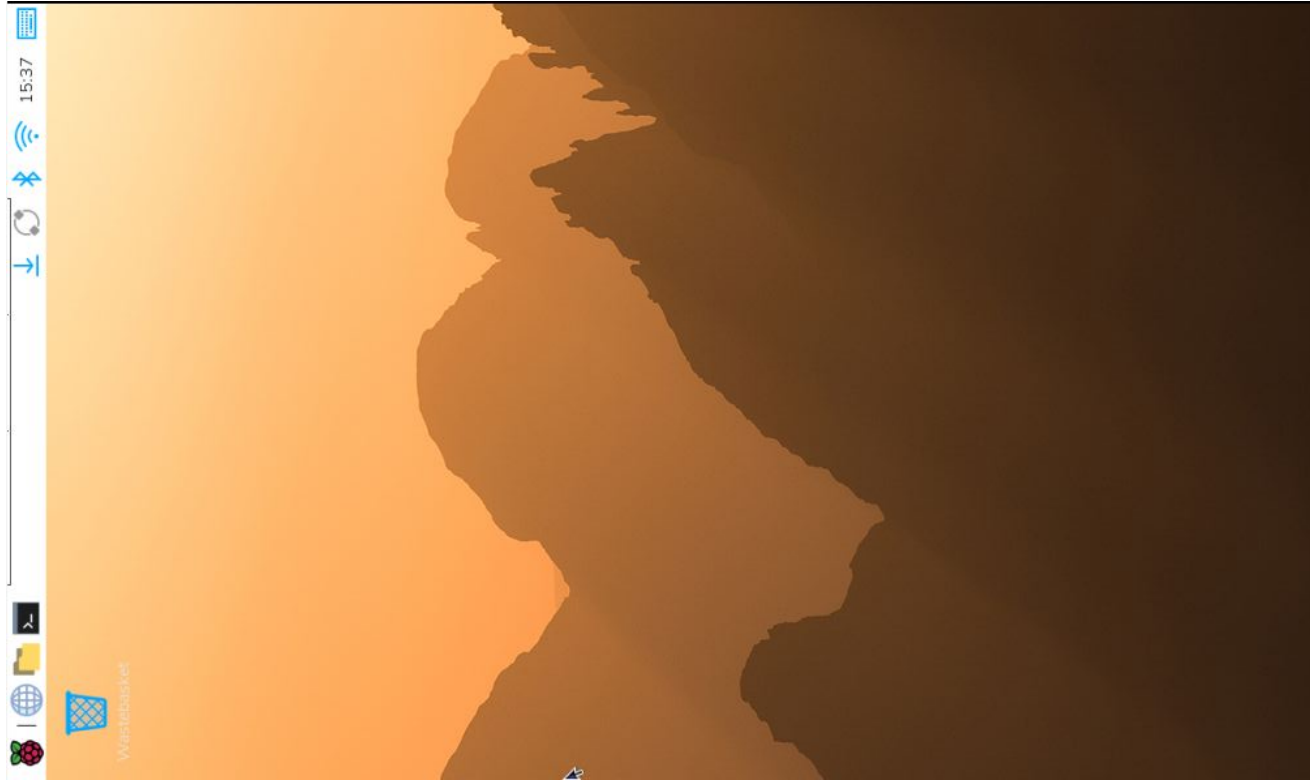
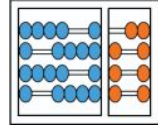
The storage device was ejected automatically. You can now remove it safely.

WRITE ANOTHER

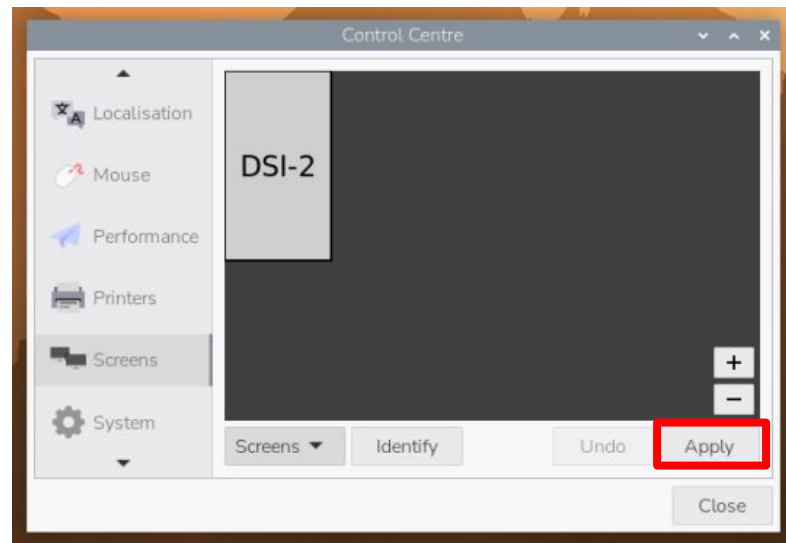
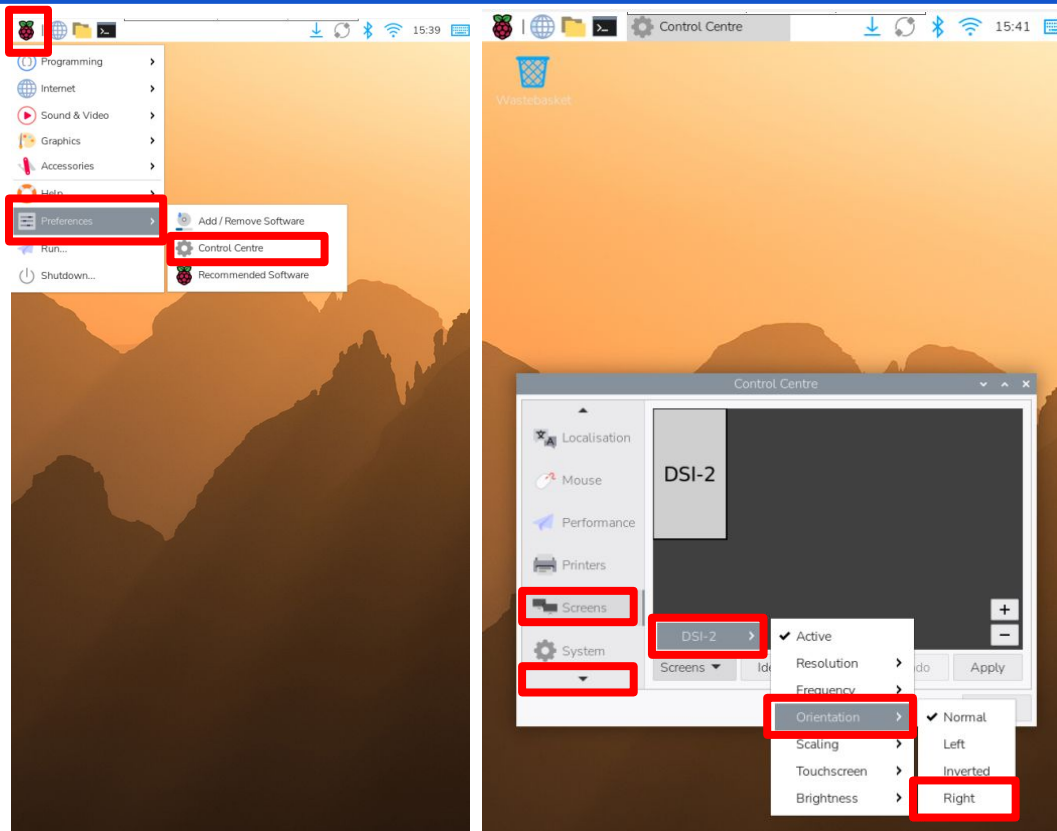
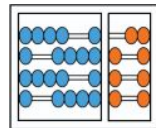
FINISH

Prática - Primeiro boot do kit

Tela inicial pós boot

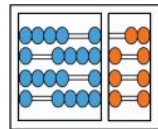


Tela inicial pós boot - Corrigindo



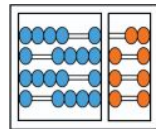
Prática - Acessando remotamente o RPI OS

Sobre o SSH



- O protocolo Secure Shell (SSH) estabelece conexões remotas criptografadas entre computadores. (Cloudfare)
 - Acesso Remoto a Servidores
 - Transferência Segura de Arquivos
 - Gerenciamento de Repositórios Git
 - Execução de comandos remotamente
 - Gerenciamento de Dispositivos IoT e de Rede
 - Encaminhamento de Gráficos
 - Etc

Checando SSH Client - Windows



- Verificando a existência

- Abrir PowerShell como Administrador
- `Get-WindowsCapability -Online | Where-Object Name -like 'OpenSSH.Client*'`

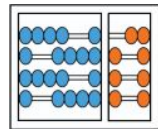
```
PS C:\WINDOWS\system32> Get-WindowsCapability -Online | Where-Object Name -like 'OpenSSH.Client*'

Name : OpenSSH.Client~~~~0.0.1.0
State : Installed
```

- Se não existir:

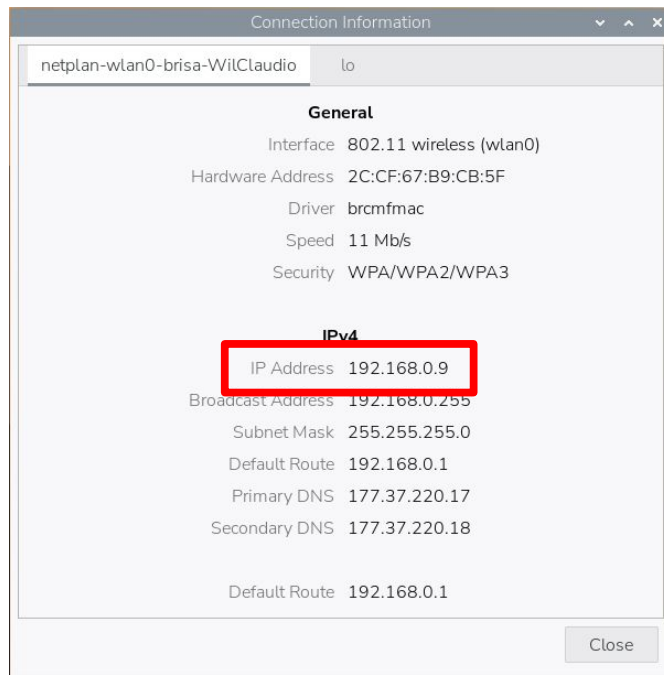
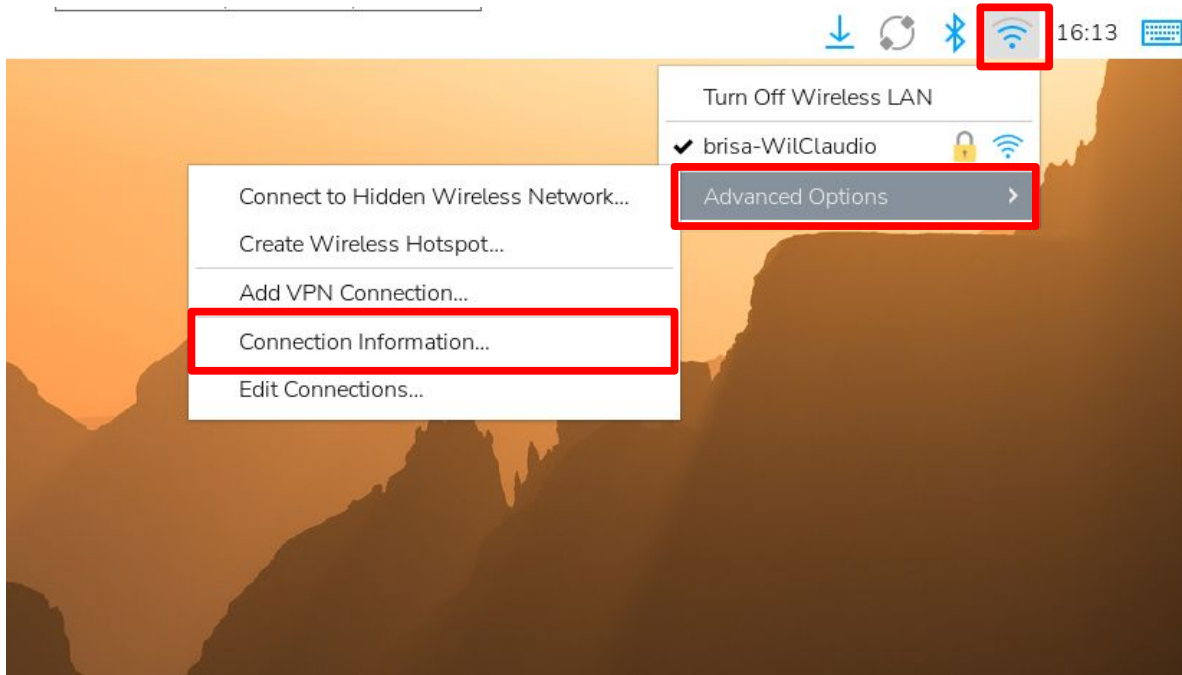
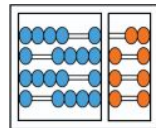
- `Add-WindowsCapability -Online -Name OpenSSH.Client~~~~0.0.1.0`

Checando SSH Client - Linux

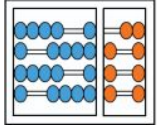


- Verificando a existência
 - `ssh --help`
- Se não existir:
 - `sudo apt update`
 - `sudo apt install openssh-client`

Checando IP - Interface Gráfica



Checando IP - Terminal

A screenshot of a Raspberry Pi desktop environment. The desktop background is a landscape image. In the top-left corner, there is a 'Wastebasket' icon. The top bar shows the system status with icons for network, Bluetooth, and Wi-Fi, and the time 16:17. A terminal window is open, showing the command 'ifconfig' being executed. The output shows the configuration for the 'eth0' interface, including the IP address 127.0.0.1. The terminal window has a menu bar with 'File', 'Edit', 'Tabs', and 'Help'. The terminal prompt is 'mateus@pi: ~'. The terminal output is as follows:

```
mateus@pi:~$ ifconfig
eth0: flags=4096<UP,BROADCAST,MULTICAST> mtu 1500
    ether 2c:cf:67:b9:cb:5e txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 106

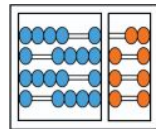
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 120 bytes 11645 (11.3 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 120 bytes 11645 (11.3 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

A close-up of the terminal output showing the configuration for the 'wlan0' interface. The output is as follows:

```
wlan0: flags=4163<UP,BROADCAST,LOOPBACK,NOARP> mtu 1500
    ether 8c:8e:f8:4d:70:00 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 106

inet 192.168.0.9 netmask 255.255.255.0
    inet6 2804:29b8:5098::1 prefixlen 64 scopeid 0x20<eth>
```

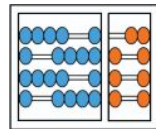
Estabelecendo conexão SSH



- Formato do comando
 - `ssh username@ip_address`
- Exemplo:
 - `ssh mateus@192.168.0.9`

```
mateus@pi: ~  
PS C:\Users\mateu> ssh mateus@192.168.0.9  
The authenticity of host '192.168.0.9 (192.168.0.9)' can't be established.  
ED25519 key fingerprint is SHA256:f1QrE3Ab3pUoaqzmYq2+lFi/GYWKa7fvuEpNRktN6dM.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '192.168.0.9' (ED25519) to the list of known hosts.  
mateus@192.168.0.9's password:  
Linux pi 6.12.47-rpt-rpi-2712 #1 SMP PREEMPT Debian 1:6.12.47-1+rpt1 (2025-09-16) aarch64  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
Last login: Fri Jan 23 16:25:52 2026 from 192.168.0.7  
mateus@pi:~ $
```


Instalação do Kotlin



- Comandos:

- `curl -s "https://get.sdkman.io" | bash`
- `source "$HOME/.sdkman/bin/sdkman-init.sh"`
- `sdk install kotlin`
- `kotlinc -version`

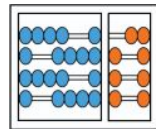
```
mateus@pi:~/Documents/codes $ kotlinc
/home/mateus/.sdkman/candidates/kotlin/current/bin/kotlinc: line 102: java: command not found
```

- Missing java!

- `sdk list java`
- `sdk install java 21.0.2-zulu`

```
mateus@pi:~/Documents/codes $ kotlinc -version
info: kotlinc-jvm 2.3.0 (JRE 21.0.2+13-LTS)
```

Validação do Kotlin



- Criar arquivo do código:
 - **cd** ~/Documents *(Acessa o diretório Documents de qualquer lugar, usando home como referência)*
 - **mkdir** codes && **cd** codes *(cria diretório codes e acessa o diretório em seguida)*
 - **touch** [hello.kt](#) *(cria o arquivo vazio)*
 - **nano** [hello.kt](#) *(abre editor de texto de terminal para editar o arquivo)*

```
hello.kt
1 fun main() {
2     println("Hello, World!")
3 }
```

- Executar o código:
 - **kotlinc** hello.kt -include-runtime -d app.jar
 - **java** -jar app.jar

Dúvidas e Discussões